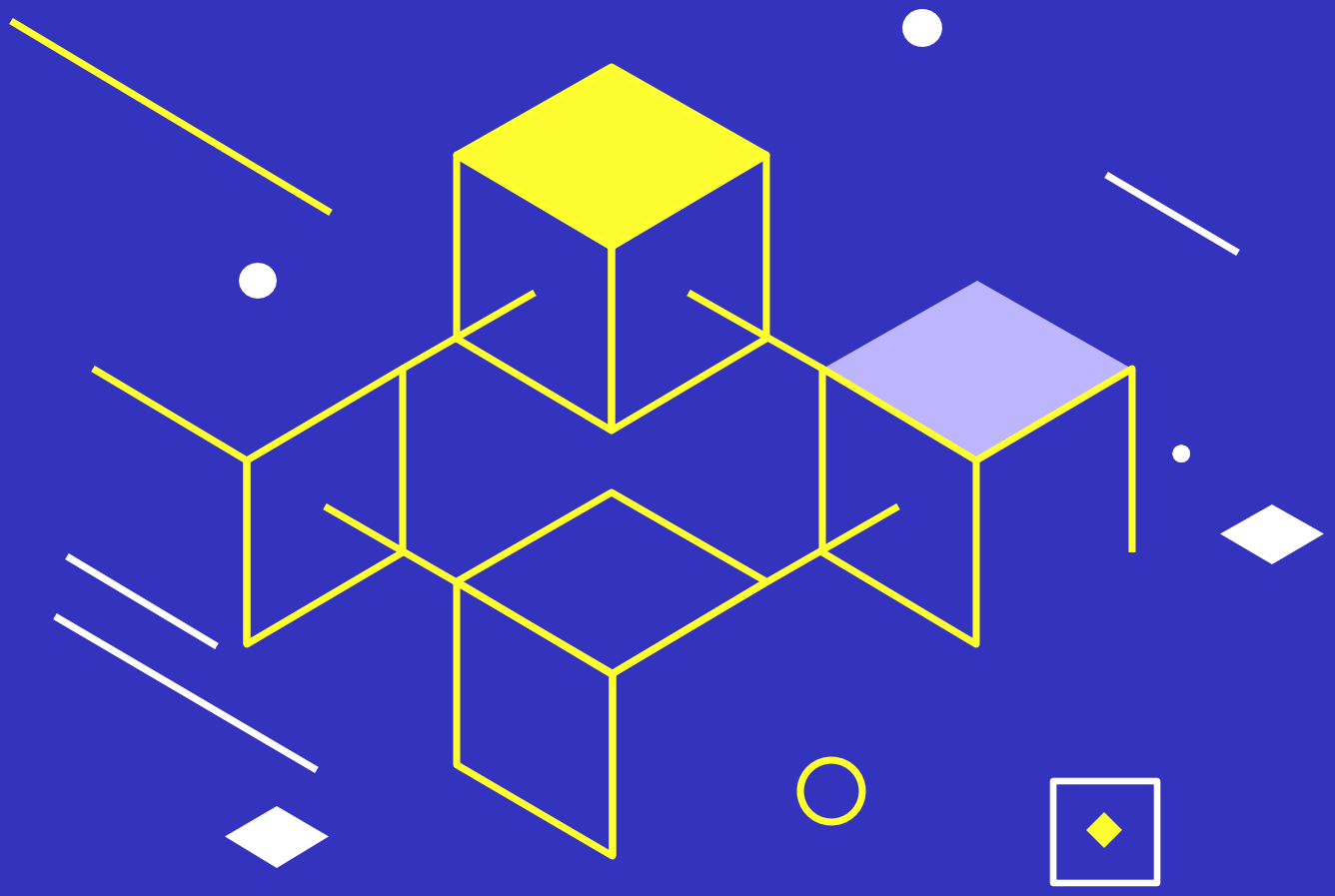
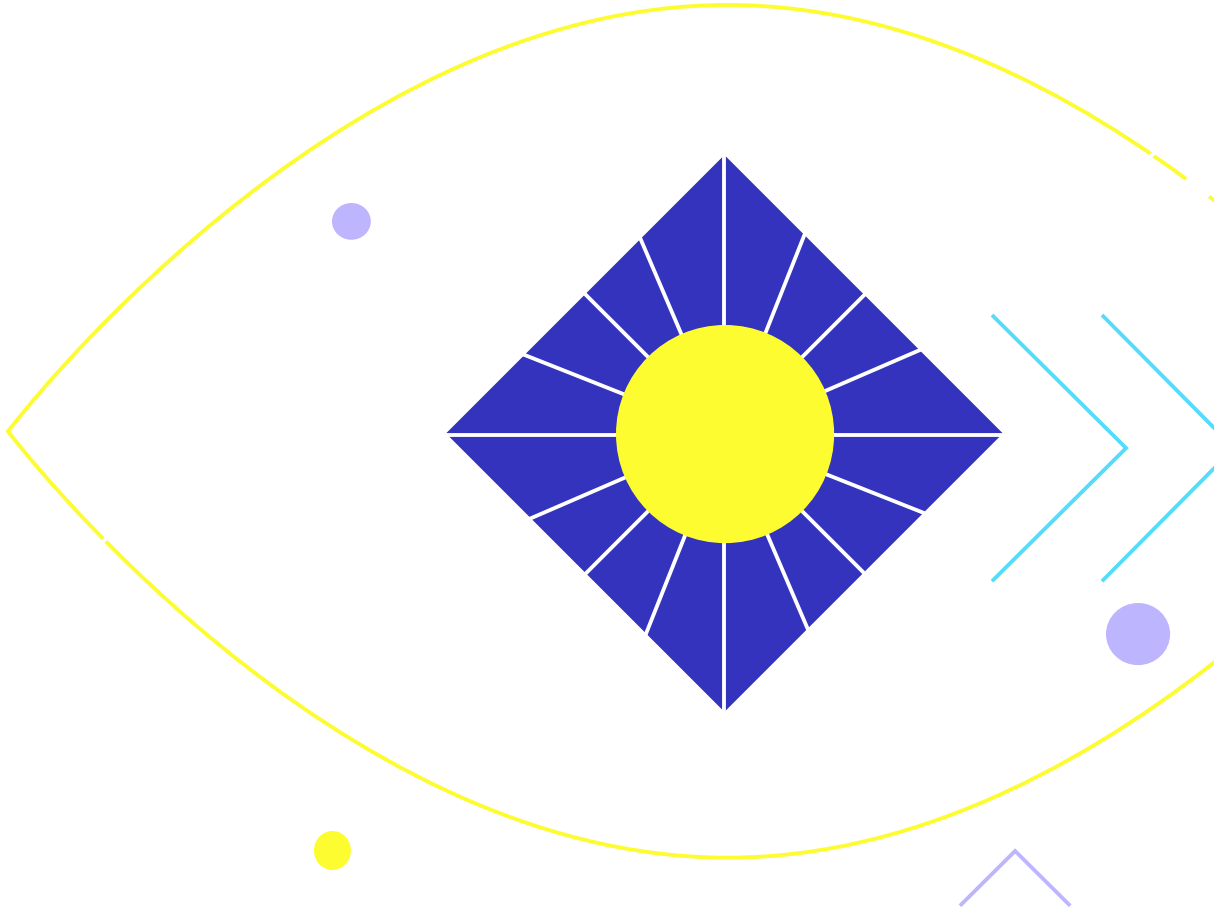
GECAP ONe

Mentoria Linux



04/12/25

The Big Picture

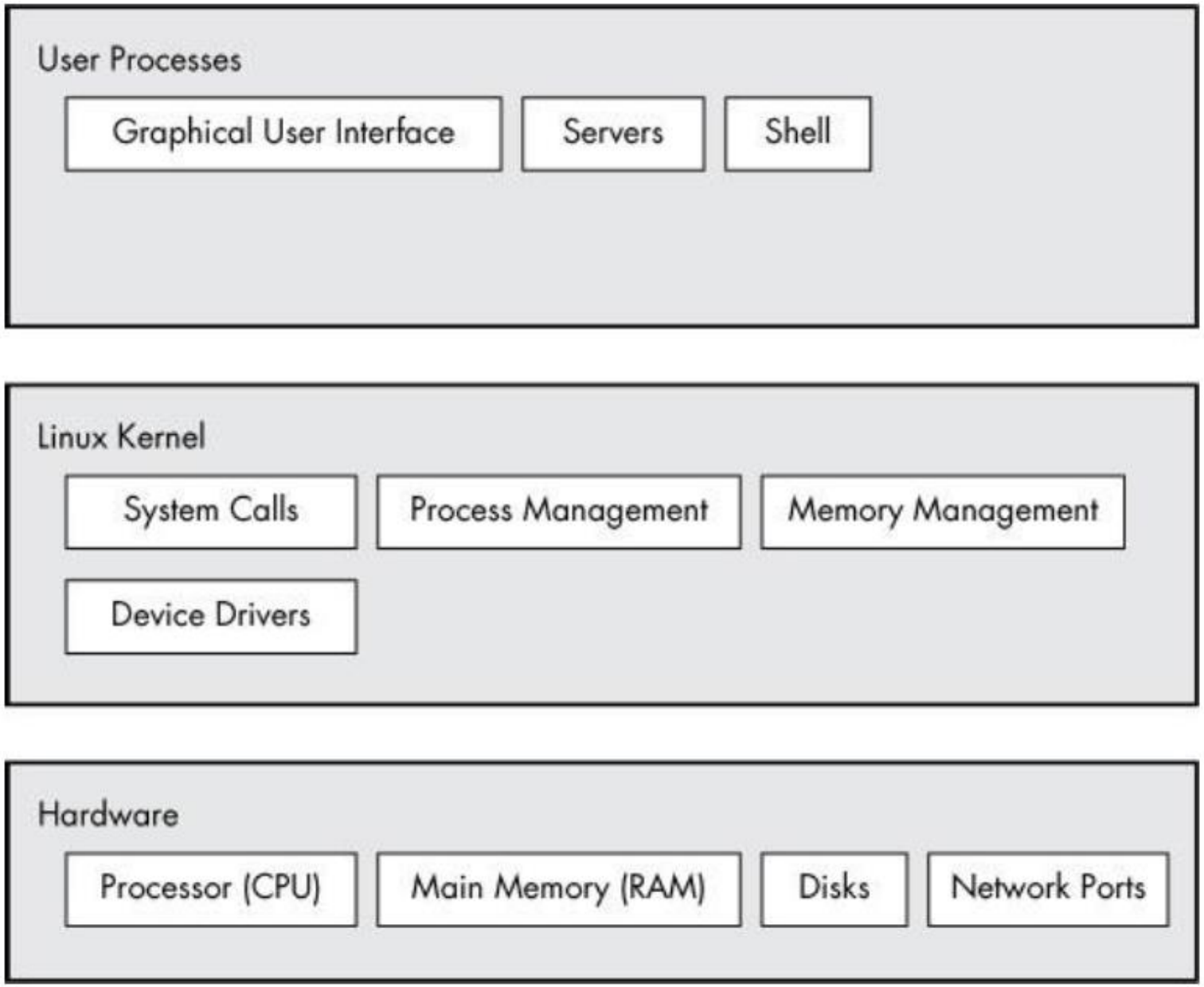


Kernel Linux

Programa residito em memória responsável por informar o CPU qual é a próxima tarefa a ser executada (e tomar as devidas providências para permitir isso).

Responsabilidades:

- Gerencia Processos
- Gerencia **memória**
- Gerencia dispositivos (drivers)
- Fornece syscalls


User Processes
Graphical User Interface
Servers
Shell
Linux Kernel
System Calls
Process Management
Memory Management
Device Drivers
Hardware
Processor (CPU)
Main Memory (RAM)
Disks
Network Ports

Kernel Linux

- O kernel determina qual processo pode usar o CPU
  - *Context switch* e *time slices*
  - *Multitasking*
  - Quando o kernel roda suas rotinas?

- O kernel faz o controle da alocação de memória e atribuí ela a um processo conforme for necessário
  - Tem sua própria região de memória **privada**
  - Cada processo precisa da sua própria seção/fatia de memória (*user space*)
  - MMU (Memory Management Unit) -> virtual memory

- O kernel executa em **kernel mode** (acesso irrestrito), processos do usuário rodam em **user mode**.

Kernel Linux

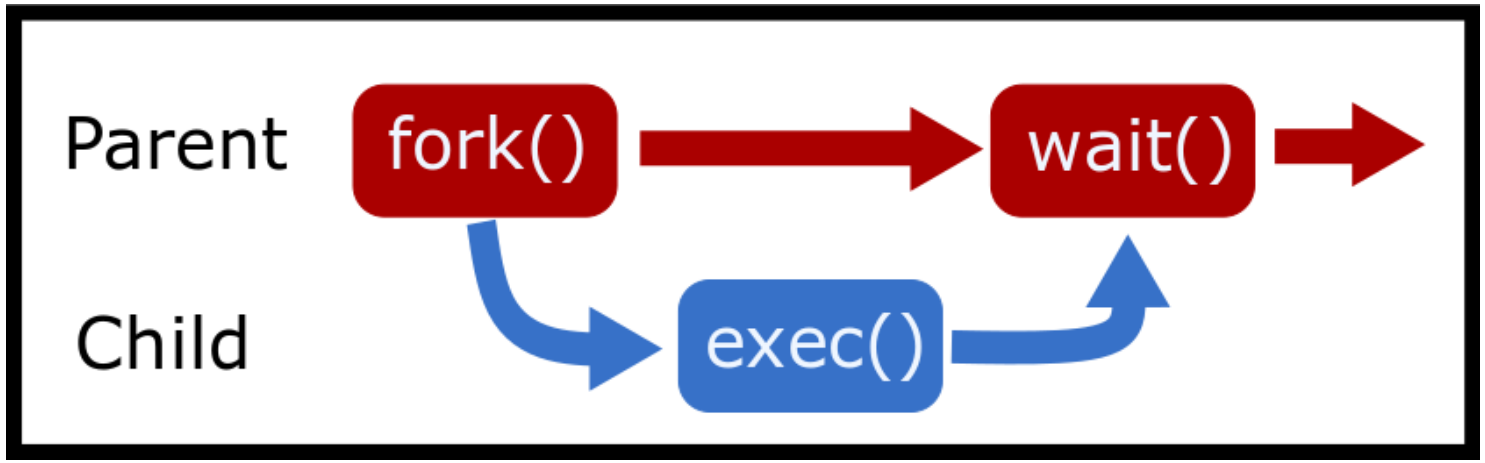
- Drivers: Módulo do kernel responsável por permitir a comunicação com algum dispositivo
  - Dispositivos com o mesmo fim provavelmente **vão ter drivers diferentes**
  - Reside no kernel, assim este consegue abstrair especificidades e expor uma interface coerente para os processos

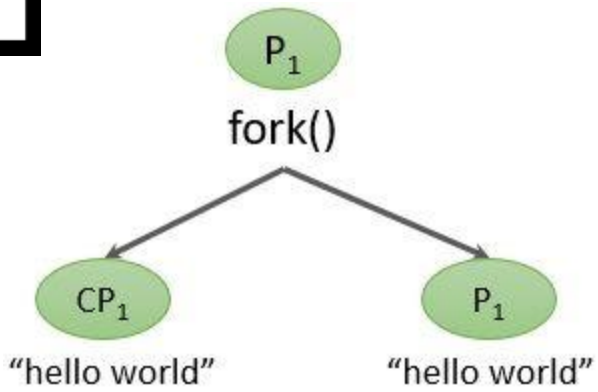
Kernel Linux

- System Calls (syscalls): Funções especializadas oferecidas pelo kernel para realizar ações que o processo não conseguiria por conta própria
  - Manipulação de Arquivos
  - Criação de processos: fork() -> exec()

- *Users*: Abstração UNIX representando *ownership* de processos e arquivos.

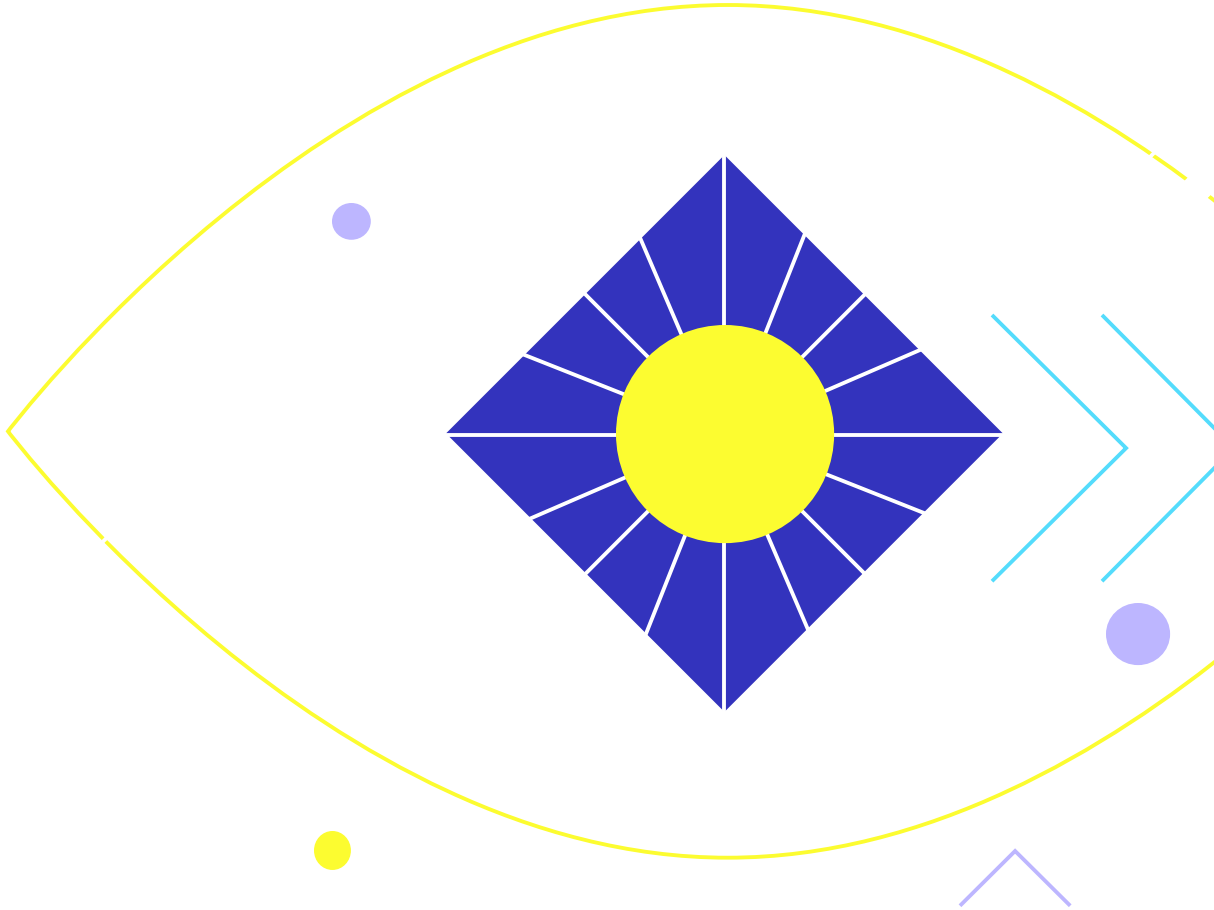
Kernel Linux





GECAP ONe

Shell e alguns comandos essenciais



Shell

- **Precursor** -> **sh** *(Bourne shell)*
  - Criado por Stephen Bourne, incorporado ao Unix 7 em 1979, Bell Labs
  - Herdado pelos sistemas UNIX-like
  - POSIX (Portable Operating System Interface)

- **Sucessor** e mais disseminado nos dias de hoje -> **bash** *(Bourne Again Shell)*
  - *Criado por* Brian Fox em 1989, para o projeto GNU
  - Proposta de ser uma alternativa completa e gratuita ao ` sh`

- Curiosidade:
  - Primeiro shell UNIX criado foi o **Thompson shell**, introduzido na 1ª versão do Unix em 1971, na Bell Labs
    - Criador: Ken Tompson

Shell

- Qual a diferença entre “Terminal” e “Shell”?

Shell

- Qual a diferença dos símbolos “#” e “$” na linha de comando do Shell?

Shell

- `cat /etc/passwd`

Shell

- `echo`
- `ls`
- `cp`
- `mv`
- `touch`
- `rm`

- Escolha um editor de texto: `vi`, `vim`, `nano`, etc.

Shell

- Hands-on: navegando pelos diretórios
  - `.`, `..`, `/`, `~`
  - `cd`, `pwd`, `mkdir`, `rmdir`
  - ***wildcards***

Shell

- Comandos intermediários
  - `**grep**`
  - `**less**`
  - `diff`
  - `file`
  - `**find**` e `locate`
  - `head` e `tail`
  - `sort`

Shell

- Mudar senha: `passwd`
- Mudar shell: `chsh`

Shell

- Como conseguir ajuda?
  - `man` e `info`
  - Google
  - Fóruns (e.g., stackoverflow)

- Extras ☺
  - die.net
  - cht.sh

Shell

- Variáveis do shell e *environment*
  - *PS1 e RPS1*
  - PATH
  - `$<var>` vs `export $<var>`

Shell

- Gerenciamento de Processos
  - ps
  - top, htop, btop

- Processos em segundo plano
  - <C-z>
  - fg
  - jobs

Shell

- Comprimindo e Extraindo arquivos
  - tar
  - gzip

Shell

- Logs do sistema
  - journalctl
    - journalctl –F _SYSTEMD_UNIT
    - journalctl –N
    - journalctl –u <service>
    - journalctl –g ‘<regex>’
    - journalctl –b -1
    - journalctl –f

Shell

- Agendando execução de tarefas
  - cron
    - crontab –e
    - crontab –l
    - crontab -r
  - systemd timer

Shell – Desafios

Regras:

- Usar comandos CLI (GNU e companhia, python não vale);
- Se não especificado, scripts devem ler do *stdin* e escrever no *stdout;*
- IA somente para consulta.

1. Instalar a linguagem Go e tornar esta disponível no Shell.

Shell – Desafios

2. Manipular dataset de Super Heróis, printar relatório no terminal
   - 1º relatório: Pegar todos os heróis do tipo "Android" e exibir "Nome, Idade, Gênero, Publicadora“
   - 2º relatório: Da lista do 1º relatório, fazer uma segunda lista separando entre *DC* e *Marvel*

   - O relatório deve ser apresentado de forma tabular, onde cada coluna é separado por espaços ou tabs e justificados à direita (ou à esquerda). Exemplo: `` `ps  aux` ``

   - Dataset: https://www.kaggle.com/datasets/claudiodavi/superhero-set
     - Exemplo de comando para download: `wget https://www.kaggle.com/api/v1/datasets/download/claudiodavi/superhero-set`

Shell – Desafios

3. Gerar relatórios periódicos da situação do clima na Sede III

- Consultar de **5 em 5 minutos** e salvar as informações no arquivo de log *weather.log* na pasta do usuário.
- As informações mínimas são: Temperatura atual, Probabilidade de chover na hora atual (ex: se agora são 15h52, então o intervalo a considerar é 15h-16h).

- Sugestão de API de clima: https://open-meteo.com/en/docs
  - Outras opções: https://github.com/public-apis/public-apis?tab=readme-ov-file#weather
- Coordenadas: -15,798501; -47,882590 (https://https://maps.google.com/)

Linux Filesystem Hierarchy Standard - FHS

O **Filesystem Hierarchy Standard** (padrão para sistema de arquivos hierárquico), ou **FHS**, define os principais diretórios, e o seu conteúdo, em um sistema operacional Linux ou do tipo Unix. A versão atual é a 3.0, anunciada em 3 de junho de 2015.

O FHS é mantido pela Linux Foundation.

Fonte: Filesystem Hierarchy Standard – Wikipédia, a enciclopédia livre